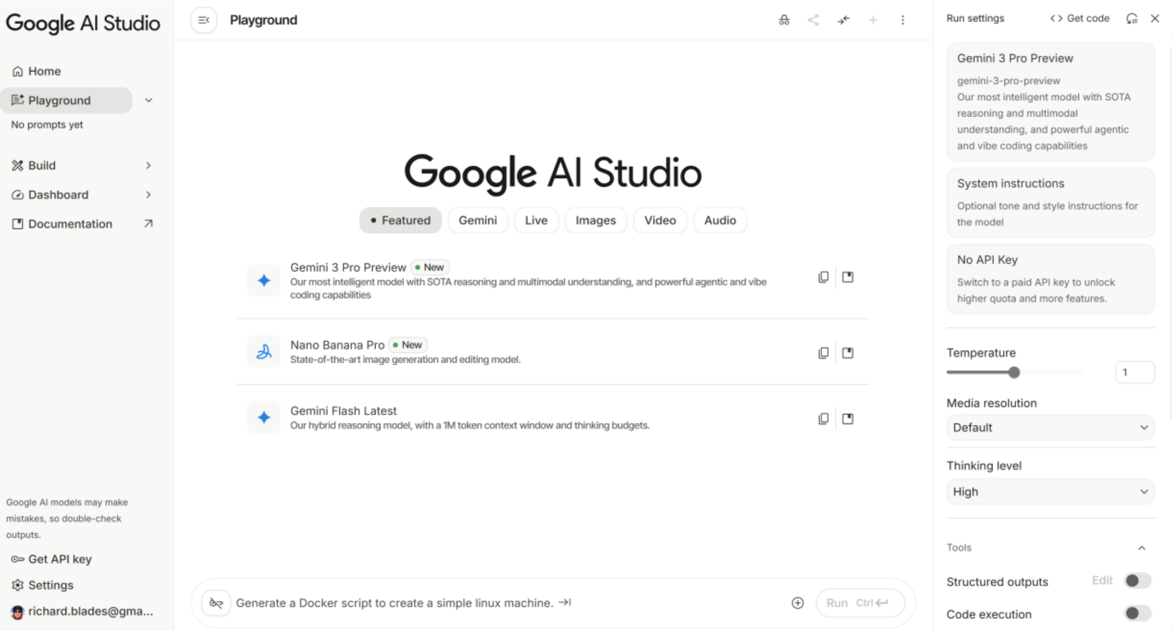
BAYES BUSINESS SCHOOL
CITY ST GEORGE'S UNIVERSITY OF LONDON

Introduction to programmatic prompting using LLM APIs

Week 9

[bayes.city.ac.uk]

Two main ways to access LLMs (from Week 8)



Via chat interface
(Week 8)

```
client = Groq(api_key=os.getenv("GROQ_API_KEY"))

completion = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[{"role": "user", "content": "Explain cost of equity in one sentence."}]
)

print(completion.choices[0].message.content)
```

Programmatically,
via APIs
(Week 9)

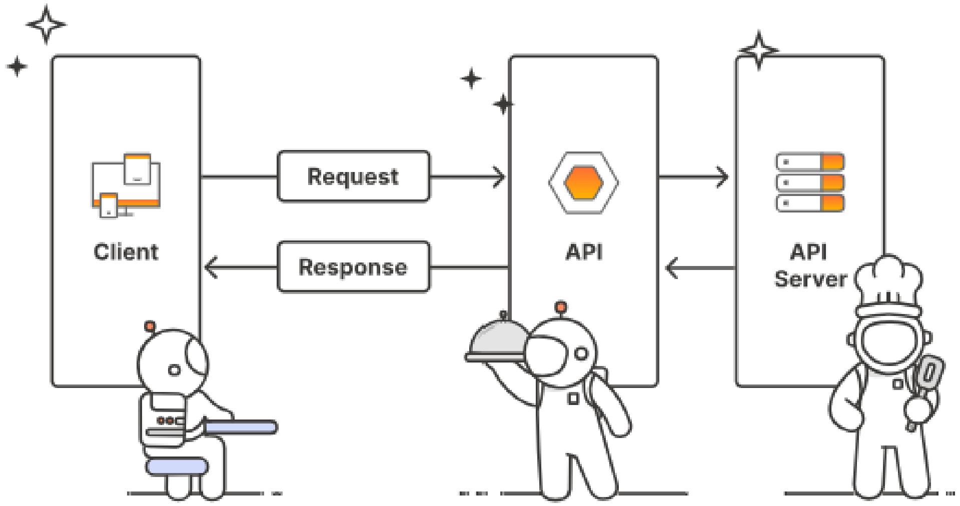
Preface

- An API is just a ***different mechanism*** of accessing the same (or more) models
- The ***same challenges around hallucinations*** still apply
- The ***same solutions*** (Week 8) to mitigate hallucinations ***still apply***
- ***Some additional solutions*** (e.g. seeding, pinned models) are ***only available via API***

BAYES
BUSINESS SCHOOL
CITY, UNIVERSITY OF LONDON

Introduction to APIs

- **A**pplication **P**rogramming **I**nterface
- A software-based interface which allows different software components to communicate and transfer data
- Key benefits:
  - ***Automation/ integration***: Allow machines to talk to machines without manual input; connect multiple systems
  - ***Reusability/efficiency***: Avoids reinventing the wheel by reusing ready-made functions
  - ***Scalability***: Enables modular, distributed systems that can grow easily



BAYES
BUSINESS SCHOOL
CITY, UNIVERSITY OF LONDON

Examples of APIs

stripe

Embedding Card Payment into a Check-out process

Google Maps

Embedding maps into a website

X Developer Platform

Accessing tweets for customer analytics

Introduction to LLM APIs

- LLM APIs are APIs which provide an interface to LLMs such as ChatGPT, Gemini, Groq, etc.
- Different APIs for ***different tasks***
  - Content types (text, audio, video), processing (e.g., reasoning), volume of data, and latency
  - Some providers even offer APIs tailored to specific use cases
- ‘***Tokens***’ (and ~volume in general) are an important consideration when using LLM APIs
  - Pricing, latency, caps on free accounts [Groq example]
- Key ***considerations*** for using LLM APIs
  - Consider your ***use case***: how does this affect rate limits and volume quotas
  - Manage ***costs***
  - ***Security***
  - ***Optimise*** and refine

BAYES BUSINESS SCHOOL
CITY, UNIVERSITY OF LONDON

Example of Groq models

AI MODEL	CURRENT SPEED (TOKENS PER SECOND)	INPUT TOKEN PRICE (PER MILLION TOKENS)	OUTPUT TOKEN PRICE (PER MILLION TOKENS)
GPT OSS 20B 128k	1,000 TPS	$0.075 (13.3M / $1)*	$0.30 (3.33M / $1)*
GPT OSS Safeguard 20B	1,000 TPS	$0.075 (13.3M / $1)*	$0.30 (3.33M / $1)*
GPT OSS 120B 128k	500 TPS	$0.15 (6.67M / $1)*	$0.60 (1.66M / $1)*
Kimi K2-0905 1T 256k	200 TPS	$1.00 (1M / $1)*	$3.00 (333,333 / $1)*
Llama 4 Scout (17Bx16E) 128k	594 TPS	$0.11 (9.09M / $1)*	$0.34 (2.94M / $1)*
Llama 4 Maverick (17Bx128E) 128k	562 TPS	$0.20 (5M / $1)*	$0.60 (1.6M / $1)*
Qwen3 32B 131k	662 TPS	$0.29 (3.44M / $1)*	$0.59 (1.69M / $1)*
Llama 3.3 70B Versatile 128k	394 TPS	$0.59 (1.69M / $1)*	$0.79 (1.27M / $1)*
Llama 3.1 8B Instant 128k	840 TPS	$0.05 (20M / $1)*	$0.08 (12.5M / $1)*

https://groq.com/pricing

BAYES BUSINESS SCHOOL
CITY, UNIVERSITY OF LONDON

Key benefits and challenges of using LLM APIs (vs. chat interface)

- Key benefits
  - ***Scale/ speed***: Faster at handling 10s/100s/1000s of requests at time
  - ***Automation***: Less (no) chance of mistakes due to human error
  - ***Customisation***: Greater opportunity to configure models to own needs (more hyperparameters, seeding, pinned models)
  - ***Greater choice of models***: More models are available via API
  - ***Portability, independence***: Easy to switch models; your 'instructions' are abstracted to your code
- Key challenges
  - ***Some coding*** (confidence) is required to work with them
  - ***Setup time***: Time required to write/ optimise code
  - ***Costs***: APIs can be expensive if used unfettered
  - Be aware of ***security*** vulnerabilities

Case Study: Deriving corporate disclosure sentiment

Overview

Work with real extracts from annual reports (for 2024) from several UK firms' forward-looking statements – covering growth plans, risks, and outlook. Use LLM tools to reveal ***hidden signals*** in firms' narratives.

Persona & Task

You are a junior investment analyst preparing insights for your portfolio manager. Your job is to ***compare several firms*** and identify which ones appear ***more (or less) confident*** about the year ahead (the '***sentiment***' of the statements)

Your colleague has extracted forward-looking guidance sentences from each company's annual report (using some of the techniques we discussed in Week 7).

Task

Determine the overall sentiment of the extracted text for each company.

Code workflow overview

1. **Initialisation and security**: Loads the data and prepares the AI by specifying the model and its settings
2. **Main processing loop** (firm-by-firm):
   - Normalise text
   - Generates the prompt and calls the API
   - Parses the answer
   - Calculates the final sentiment score
3. **Saves and displays** the output

Code workflow (1)

1. **Initial Setup**

   - **Loads the Data**: Reads a *pickle* file containing company names, years, and their forward-looking statements.
   - **Security Check**: Retrieves the API key. If the key is missing, the program stops immediately.
   - **Prepares the AI**: Sets the "temperature" (creativity level) to 0 to ensure the AI gives us consistent factual answers rather than creative ones.
   - **Loads Few-shot Examples**: Loads examples of what "positive“, “neutral” and "negative" sentences are like for the AI to use as a reference point.

Code workflow (2)

2. **Main processing loop (firm-by-firm)**

- **Cleans the text**. sentences = normalise_forward_sentences(data).
  - If no sentences are found, mark everything as () and move to the next firm.
- **Calls the LLM API**: Constructs a “prompt" containing the instructions and the sentences, and sends this to the AI model.
  - “Retry" logic: if the AI fails, it waits 2 seconds and tries again (up to 3 times)
- **Parses the answer**: Finds the JSON data in the AI's response.
  - Converts the labels ("positive“, etc.) into numbers (1, 0, -1). Extracts the AI’s rationale.
- **Calculates firm measure**: Calculates the mean sentiment score based on the scores of the individual sentences

Code workflow (3)

3. **Saves and displays table**
   - **Combines results:** Attaches the sentiment scores back to the original company information.
   - **Data formatting**: Ensures all numbers are stored correctly (not as text).
   - **Saves**: Exports the final results to a .CSV and a pickle file and displays final results table.